

# AUTOMOBILE SERVICE CENTRE MANAGEMENT SYSTEM

---

## **A Complete C Programming Project Report**

Using Dynamic Memory Allocation, Insertion Sort and Text File Handling

Submitted by: \_\_\_\_\_Nithesh(192521305)\_, Jatin Vijaykumar(192412047)\_\_\_\_\_

Institution: \_\_\_\_\_SAVEETHA SCHOOLE OF ENGINEERING\_\_\_\_\_

## CERTIFICATE

This is to certify that the project entitled “Automobile Service Centre Management System” is a bona fide work prepared as part of the C Programming coursework. The project demonstrates the use of structures, functions, pointers, dynamic memory allocation, searching, sorting, validation and file handling in the C programming language.

Guide Signature: \_\_\_\_\_

Student Signature: \_\_\_\_\_NITHESH.Y\_\_\_\_\_

Date: \_\_\_\_\_

## **ACKNOWLEDGEMENT**

I would like to express my sincere gratitude to my faculty member for providing guidance and encouragement during the development of this project. I also thank my institution for providing the resources and learning environment necessary to complete this work. This project helped me understand the practical application of fundamental C programming concepts such as structures, functions, pointers, dynamic memory allocation, sorting algorithms and file handling.

## ABSTRACT

The Automobile Service Centre Management System is a menu-driven application developed using the C programming language. The purpose of the system is to digitize the basic operations of an automobile service centre, including vehicle registration, storage of customer details, service cost calculation, record searching, updating, sorting and daily revenue reporting.

Unlike a fixed-size implementation, this project uses dynamic memory allocation. The program begins with no preallocated record array and expands its memory whenever a new vehicle is registered. The `realloc()` function is used for this purpose. Vehicle records are represented using a structure that combines textual and numerical information into a single logical entity.

The application validates important user inputs, prevents duplicate vehicle numbers, rejects negative costs and stores data permanently in a text file. Insertion Sort is used to arrange vehicle records according to their total service cost. A comprehensive test plan is included to verify normal operations, boundary conditions and invalid inputs.

The project demonstrates how fundamental C programming concepts can be integrated to create a practical management application.

### Keywords

C Programming, Automobile Service Centre, Structures, Dynamic Memory, Pointers, Insertion Sort, File Handling, Input Validation.

## **TABLE OF CONTENTS**

1. Problem Statement
2. Objectives
3. Scope of the Project
4. Requirements and Development Environment
5. System Analysis
6. Proposed Solution and System Design
7. Data Structure Design
8. Algorithms and Pseudocode
9. Explanation of Major Modules
10. Source Code
11. Testing and Test Cases
12. Sample Test Data and Output
13. Results and Discussion
14. Conclusion
15. Future Enhancements
16. References

## 1. PROBLEM STATEMENT

Traditional automobile service centres frequently depend on handwritten registers or basic manual records to manage vehicle servicing information. This method becomes difficult when the number of customers increases. Searching old records requires manual effort, calculations may contain errors and records can be duplicated or lost.

The assignment source identifies operational inefficiency, financial calculation errors, slow record retrieval, duplicate entries and weak persistence as important problems in a manual service-register environment. The proposed solution is a C-based computerized system for managing vehicle and service information. [filecite\turn2file0\L39-L46](#)

The system developed in this project addresses these requirements through a menu-driven application. It stores vehicle information, calculates service charges automatically, allows records to be searched and updated, produces summaries and saves data for later use.

## 2. OBJECTIVES

- To create a computerized system for maintaining automobile service records.
- To register vehicle and customer information.
- To prevent duplicate vehicle numbers.
- To calculate the total service cost automatically.
- To search for an existing vehicle record.
- To update selected vehicle and service details.
- To sort records according to total service cost.
- To display records above a user-specified service amount.
- To generate a daily service summary and total revenue.
- To store and retrieve records using file handling.
- To validate dates and financial input values.

### 3. SCOPE OF THE PROJECT

The system is designed for small or medium-scale automobile service centres. It manages the core information required for a single service record and provides a console-based interface. The program is intended for educational and academic use and demonstrates procedural programming in C.

The current scope includes registration, searching, updating, billing, sorting, filtering, daily summaries and persistent file storage. The project does not currently include a graphical user interface, multiple user accounts, network access or database integration.

### 4. REQUIREMENTS AND DEVELOPMENT ENVIRONMENT

#### 4.1 Software Requirements

- Programming Language: C (C99 compatible)
- Compiler: GCC
- IDE: Visual Studio Code, Code::Blocks or any standard C IDE
- Libraries: stdio.h, stdlib.h and string.h

#### 4.2 Hardware Requirements

- A standard computer with a C compiler
- Minimal RAM and storage space

#### 4.3 Input and Output

Input is accepted through the keyboard. Output is displayed on the console. Vehicle records are stored in a text file named vehicle\_data.txt.

The assignment source similarly specifies C, GCC, standard libraries and a console-oriented environment. □filecite□turn2file0□L47-L61□

## 5. SYSTEM ANALYSIS

### 5.1 Existing Approach

A manual register may contain customer information, vehicle details, service descriptions and costs. The information must be searched manually, and calculations are usually performed by a person. This approach can be slow and susceptible to mistakes.

### 5.2 Limitations

- Slow searching of records
- Possibility of duplicate entries
- Manual calculation errors
- Difficulty in producing daily summaries
- Risk of losing paper records
- Difficulty in sorting and analysing service costs

### 5.3 Proposed System

The proposed system automates these activities through functions. It also uses defensive validation to reject invalid costs and duplicate vehicle numbers. The source assignment specifically emphasizes duplicate prevention, validation and safe file handling as important data-integrity requirements.

□filecite□turn2file0□L80-L87□

## 6. PROPOSED SOLUTION AND SYSTEM DESIGN

The program follows a modular procedural design. Every major operation is implemented as a separate function. The central data object is the Vehicle structure.

A key difference in this implementation is dynamic memory management. Instead of defining a fixed array with a maximum capacity, the program stores records through a Vehicle pointer. When a new record is added, `realloc()` increases the memory block. This approach makes the implementation structurally different from a static-array approach while still satisfying the functional requirements.

### 6.1 Main Functional Modules

- Add Vehicle
- Search Vehicle
- Update Vehicle
- Generate Bill
- Sort Vehicles by Cost
- Display Vehicles Above Amount
- Daily Service Summary
- Display All Vehicles
- Save Records
- Load Records

### 6.2 Design Flow

Start → Load existing records → Display menu → Read user choice → Execute selected module → Return to menu → Save records and release memory on exit → Stop.

## 7. DATA STRUCTURE DESIGN

A structure is used to keep all information related to one service record together. This is consistent with the assignment's requirement to use a struct for representing diverse vehicle information as a unified entity. □ filecite □ turn2file0 □ L62-L69 □

```
typedef struct
{
    char vehicleNumber[20];
    char customerName[50];
    char vehicleModel[50];
    char serviceType[50];
    char serviceDate[15];
    float partsCost;
    float labourCost;
    float totalCost;
} Vehicle;
```

### Field Description

- vehicleNumber: Unique identification number of the vehicle.
- customerName: Name of the customer.
- vehicleModel: Model or type of vehicle.
- serviceType: Description of the service performed.
- serviceDate: Date of service in DD-MM-YYYY format.
- partsCost: Cost of replacement parts.
- labourCost: Labour charge.
- totalCost: Sum of partsCost and labourCost.

## 8. ALGORITHMS AND PSEUDOCODE

### 8.1 Main Algorithm

```
START
  Load records from file
  REPEAT
    Display menu
    Read choice
    IF choice is 1, add vehicle
    ELSE IF choice is 2, search vehicle
    ELSE IF choice is 3, update vehicle
    ELSE IF choice is 4, generate bill
    ELSE IF choice is 5, sort vehicles
    ELSE IF choice is 6, display records above amount
    ELSE IF choice is 7, generate daily summary
    ELSE IF choice is 8, display all records
    ELSE IF choice is 9, save records
    ELSE IF choice is 0, save and exit
    OTHERWISE display invalid choice
  UNTIL choice equals 0
  Release allocated memory
STOP
```

### 8.2 Add Vehicle Algorithm

```
START
Read vehicle number
Search for duplicate number
IF duplicate exists
  Display error and return
ENDIF
Read customer and vehicle details
Validate service date
Read and validate costs
Calculate total cost
Expand memory using realloc()
Store new record
Increase record count
STOP
```

### 8.3 Sorting Algorithm

Insertion Sort is used. The algorithm selects one record at a time and inserts it into the correct position among the previously sorted records. Its worst-case time complexity is  $O(n^2)$ , but it is simple and appropriate for a small educational record-management system.

## 9. EXPLANATION OF MAJOR MODULES

### 9.1 Input Buffer Handling

`clearInputBuffer()` removes remaining characters after `scanf()`. This prevents a leftover newline from causing `fgets()` to read an empty string.

### 9.2 Cost Calculation

`calculateTotal()` receives a pointer to a `Vehicle` and updates the original record directly. The total cost is calculated as parts cost plus labour cost.

### 9.3 Duplicate Detection

`findVehicle()` performs a linear search using `strcmp()`. If the same vehicle number is already present, registration is rejected.

### 9.4 Dynamic Memory

The program uses `realloc()` when a new record is added. A temporary pointer is used so that the existing pointer is not lost if memory allocation fails.

### 9.5 File Handling

The save module writes fields separated by the `|` character. The load module reads the same format. This provides persistent storage between program executions.

### 9.6 Sorting

Insertion Sort rearranges complete `Vehicle` structures according to `totalCost` in ascending order.

## 10. COMPLETE SOURCE CODE

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define FILE_NAME "vehicle_data.txt"

typedef struct
{
    char vehicleNumber[20];
    char customerName[50];
    char vehicleModel[50];
    char serviceType[50];
    char serviceDate[15];
    float partsCost;
    float labourCost;
    float totalCost;
} Vehicle;

void clearInputBuffer(void)
{
    int ch;
    while ((ch = getchar()) != '\n' && ch != EOF) { }
}

void calculateTotal(Vehicle *v)
{
    if (v != NULL)
        v->totalCost = v->partsCost + v->labourCost;
}

int findVehicle(const Vehicle *vehicles, int count, const char *number)
{
    int i;
    for (i = 0; i < count; i++)
        if (strcmp(vehicles[i].vehicleNumber, number) == 0)
            return i;
    return -1;
}

int validateDate(const char *date)
{
    int day, month, year;

    if (strlen(date) != 10 || date[2] != '-' || date[5] != '-')
        return 0;

    if (sscanf(date, "%2d-%2d-%4d", &day, &month, &year) != 3)
        return 0;

    if (day < 1 || day > 31 || month < 1 || month > 12)
        return 0;

    if (year < 2000 || year > 2100)
        return 0;

    return 1;
}

float getValidCost(const char *message)
{
    float cost;

    while (1)
```

```

{
    printf("%s", message);

    if (scanf("%f", &cost) != 1)
    {
        printf("Invalid input! Enter a numeric value.\n");
        clearInputBuffer();
        continue;
    }

    clearInputBuffer();

    if (cost < 0)
    {
        printf("Cost cannot be negative.\n");
        continue;
    }

    return cost;
}

}

void displayVehicle(const Vehicle *v)
{
    printf("\n-----\n");
    printf("Vehicle Number : %s\n", v->vehicleNumber);
    printf("Customer Name : %s\n", v->customerName);
    printf("Vehicle Model : %s\n", v->vehicleModel);
    printf("Service Type : %s\n", v->serviceType);
    printf("Service Date : %s\n", v->serviceDate);
    printf("Parts Cost : Rs. %.2f\n", v->partsCost);
    printf("Labour Cost : Rs. %.2f\n", v->labourCost);
    printf("Total Cost : Rs. %.2f\n", v->totalCost);
    printf("-----\n");
}

void addVehicle(Vehicle **vehicles, int *count)
{
    Vehicle newVehicle;
    Vehicle *temp;

    printf("\n==== ADD VEHICLE ==== \n");

    printf("Enter Vehicle Number: ");
    scanf("%19s", newVehicle.vehicleNumber);
    clearInputBuffer();

    if (findVehicle(*vehicles, *count, newVehicle.vehicleNumber) != -1)
    {
        printf("Duplicate vehicle number!\n");
        return;
    }

    printf("Enter Customer Name: ");
    fgets(newVehicle.customerName, sizeof(newVehicle.customerName), stdin);
    newVehicle.customerName[strcspn(newVehicle.customerName, "\n")] = '\0';

    printf("Enter Vehicle Model: ");
    fgets(newVehicle.vehicleModel, sizeof(newVehicle.vehicleModel), stdin);
    newVehicle.vehicleModel[strcspn(newVehicle.vehicleModel, "\n")] = '\0';

    printf("Enter Service Type: ");
    fgets(newVehicle.serviceType, sizeof(newVehicle.serviceType), stdin);
    newVehicle.serviceType[strcspn(newVehicle.serviceType, "\n")] = '\0';
}

```

```

do
{
    printf("Enter Service Date (DD-MM-YYYY): ");
    scanf("%14s", newVehicle.serviceDate);
    clearInputBuffer();

    if (!validateDate(newVehicle.serviceDate))
        printf("Invalid date format or range. Try again.\n");
} while (!validateDate(newVehicle.serviceDate));

newVehicle.partsCost = getValidCost("Enter Parts Cost: ");
newVehicle.labourCost = getValidCost("Enter Labour Cost: ");

calculateTotal(&newVehicle);

temp = realloc(*vehicles, (*count + 1) * sizeof(Vehicle));

if (temp == NULL)
{
    printf("Memory allocation failed!\n");
    return;
}

*vehicles = temp;
(*vehicles)[*count] = newVehicle;
(*count)++;

printf("Vehicle added successfully.\n");
printf("Total Service Cost: Rs. %.2f\n", newVehicle.totalCost);
}

void searchVehicle(const Vehicle *vehicles, int count)
{
    char number[20];
    int index;

    if (count == 0)
    {
        printf("No records available.\n");
        return;
    }

    printf("Enter Vehicle Number: ");
    scanf("%19s", number);
    clearInputBuffer();

    index = findVehicle(vehicles, count, number);

    if (index == -1)
        printf("Vehicle not found!\n");
    else
        displayVehicle(&vehicles[index]);
}

void updateVehicle(Vehicle *vehicles, int count)
{
    char number[20];
    int index, choice;

    if (count == 0)
    {
        printf("No records available.\n");
        return;
    }
}

```

```

printf("Enter Vehicle Number: ");
scanf("%19s", number);
clearInputBuffer();

index = findVehicle(vehicles, count, number);

if (index == -1)
{
    printf("Vehicle not found!\n");
    return;
}

printf("\n1. Customer Name\n2. Vehicle Model\n3. Service Type\n");
printf("4. Parts Cost\n5. Labour Cost\nEnter choice: ");

if (scanf("%d", &choice) != 1)
{
    printf("Invalid choice!\n");
    clearInputBuffer();
    return;
}
clearInputBuffer();

switch (choice)
{
    case 1:
        printf("Enter New Customer Name: ");
        fgets(vehicles[index].customerName,
            sizeof(vehicles[index].customerName), stdin);
        vehicles[index].customerName[
            strcspn(vehicles[index].customerName, "\n")] = '\0';
        break;

    case 2:
        printf("Enter New Vehicle Model: ");
        fgets(vehicles[index].vehicleModel,
            sizeof(vehicles[index].vehicleModel), stdin);
        vehicles[index].vehicleModel[
            strcspn(vehicles[index].vehicleModel, "\n")] = '\0';
        break;

    case 3:
        printf("Enter New Service Type: ");
        fgets(vehicles[index].serviceType,
            sizeof(vehicles[index].serviceType), stdin);
        vehicles[index].serviceType[
            strcspn(vehicles[index].serviceType, "\n")] = '\0';
        break;

    case 4:
        vehicles[index].partsCost =
            getValidCost("Enter New Parts Cost: ");
        calculateTotal(&vehicles[index]);
        break;

    case 5:
        vehicles[index].labourCost =
            getValidCost("Enter New Labour Cost: ");
        calculateTotal(&vehicles[index]);
        break;

    default:
        printf("Invalid choice!\n");
        return;
}

```

```

    printf("Vehicle details updated successfully.\n");
}

void generateBill(const Vehicle *vehicles, int count)
{
    char number[20];
    int index;

    if (count == 0)
    {
        printf("No records available.\n");
        return;
    }

    printf("Enter Vehicle Number: ");
    scanf("%19s", number);
    clearInputBuffer();

    index = findVehicle(vehicles, count, number);

    if (index == -1)
    {
        printf("Vehicle not found!\n");
        return;
    }

    printf("\n===== SERVICE BILL =====\n");
    printf("Vehicle Number : %s\n", vehicles[index].vehicleNumber);
    printf("Customer Name : %s\n", vehicles[index].customerName);
    printf("Vehicle Model : %s\n", vehicles[index].vehicleModel);
    printf("Service Type : %s\n", vehicles[index].serviceType);
    printf("-----\n");
    printf("Parts Cost : Rs. %.2f\n", vehicles[index].partsCost);
    printf("Labour Cost : Rs. %.2f\n", vehicles[index].labourCost);
    printf("-----\n");
    printf("TOTAL BILL : Rs. %.2f\n", vehicles[index].totalCost);
    printf("=====\n");
}

void sortVehicles(Vehicle *vehicles, int count)
{
    int i, j;
    Vehicle key;

    if (count < 2)
    {
        printf("Not enough records to sort.\n");
        return;
    }

    for (i = 1; i < count; i++)
    {
        key = vehicles[i];
        j = i - 1;

        while (j >= 0 && vehicles[j].totalCost > key.totalCost)
        {
            vehicles[j + 1] = vehicles[j];
            j--;
        }

        vehicles[j + 1] = key;
    }
}

```

```

    printf("Vehicles sorted by total service cost.\n");
}

void displayAboveAmount(const Vehicle *vehicles, int count)
{
    float amount;
    int i, found = 0;

    if (count == 0)
    {
        printf("No records available.\n");
        return;
    }

    amount = getValidCost("Enter Amount: ");

    for (i = 0; i < count; i++)
    {
        if (vehicles[i].totalCost > amount)
        {
            displayVehicle(&vehicles[i]);
            found = 1;
        }
    }

    if (!found)
        printf("No vehicles found above the specified amount.\n");
}

void dailySummary(const Vehicle *vehicles, int count)
{
    char date[15];
    int i, vehicleCount = 0;
    float totalRevenue = 0.0f;

    printf("Enter Date (DD-MM-YYYY): ");
    scanf("%14s", date);
    clearInputBuffer();

    if (!validateDate(date))
    {
        printf("Invalid date.\n");
        return;
    }

    printf("\n==== DAILY SERVICE SUMMARY =====\n");

    for (i = 0; i < count; i++)
    {
        if (strcmp(vehicles[i].serviceDate, date) == 0)
        {
            vehicleCount++;
            totalRevenue += vehicles[i].totalCost;
            printf("%d. %s - Rs. %.2f\n", vehicleCount,
                vehicles[i].vehicleNumber,
                vehicles[i].totalCost);
        }
    }

    printf("Total Vehicles: %d\n", vehicleCount);
    printf("Total Revenue : Rs. %.2f\n", totalRevenue);
}

void displayAllVehicles(const Vehicle *vehicles, int count)
{

```

```

int i;

if (count == 0)
{
    printf("No records available.\n");
    return;
}

printf("\n===== ALL VEHICLE RECORDS =====\n");

for (i = 0; i < count; i++)
    displayVehicle(&vehicles[i]);
}

void saveToFile(const Vehicle *vehicles, int count)
{
    FILE *file;
    int i;

    file = fopen(FILE_NAME, "w");

    if (file == NULL)
    {
        printf("Unable to open file for saving.\n");
        return;
    }

    for (i = 0; i < count; i++)
    {
        fprintf(file, "%s|%s|%s|%s|%s|%.2f|%.2f|%.2f\n",
            vehicles[i].vehicleNumber,
            vehicles[i].customerName,
            vehicles[i].vehicleModel,
            vehicles[i].serviceType,
            vehicles[i].serviceDate,
            vehicles[i].partsCost,
            vehicles[i].labourCost,
            vehicles[i].totalCost);
    }

    fclose(file);
    printf("Records saved successfully.\n");
}

void loadFromFile(Vehicle **vehicles, int *count)
{
    FILE *file;
    Vehicle temp;

    file = fopen(FILE_NAME, "r");

    if (file == NULL)
    {
        printf("No previous records found. Starting new database.\n");
        return;
    }

    while (fscanf(file,
        "%19[^|]|%49[^|]|%49[^|]|%49[^|]|%14[^|]|%f|%f|%f\n",
        temp.vehicleNumber,
        temp.customerName,
        temp.vehicleModel,
        temp.serviceType,
        temp.serviceDate,
        &temp.partsCost,

```

```

        &temp.labourCost,
        &temp.totalCost) == 8)
{
    Vehicle *tempMemory =
        realloc(*vehicles, (*count + 1) * sizeof(Vehicle));

    if (tempMemory == NULL)
    {
        printf("Memory allocation failed while loading data.\n");
        break;
    }

    *vehicles = tempMemory;
    (*vehicles)[*count] = temp;
    (*count)++;
}

fclose(file);
printf("%d record(s) loaded.\n", *count);
}

```

```

int main(void)
{
    Vehicle *vehicles = NULL;
    int count = 0;
    int choice;

    loadFromFile(&vehicles, &count);

    do
    {
        printf("\n===== \n");
        printf(" AUTOMOBILE SERVICE CENTRE SYSTEM\n");
        printf("===== \n");
        printf("1. Add Vehicle\n");
        printf("2. Search Vehicle\n");
        printf("3. Update Vehicle\n");
        printf("4. Generate Bill\n");
        printf("5. Sort Vehicles by Cost\n");
        printf("6. Display Vehicles Above Amount\n");
        printf("7. Daily Service Summary\n");
        printf("8. Display All Vehicles\n");
        printf("9. Save Records\n");
        printf("0. Exit\n");
        printf("Enter your choice: ");

        if (scanf("%d", &choice) != 1)
        {
            printf("Invalid input! Please enter a menu number.\n");
            clearInputBuffer();
            continue;
        }

        clearInputBuffer();

        switch (choice)
        {
            case 1: addVehicle(&vehicles, &count); break;
            case 2: searchVehicle(vehicles, count); break;
            case 3: updateVehicle(vehicles, count); break;
            case 4: generateBill(vehicles, count); break;
            case 5: sortVehicles(vehicles, count); break;
            case 6: displayAboveAmount(vehicles, count); break;
            case 7: dailySummary(vehicles, count); break;
            case 8: displayAllVehicles(vehicles, count); break;
        }
    }
}

```

## Automobile Service Centre Management System

```
        case 9: saveToFile(vehicles, count); break;
        case 0:
            saveToFile(vehicles, count);
            printf("Thank you for using the system.\n");
            break;
        default:
            printf("Invalid choice! Try again.\n");
    }

} while (choice != 0);

free(vehicles);
return 0;
}
```

## 11. TESTING AND TEST CASES

Testing is performed to verify that each function produces the expected result and handles invalid input safely. The following test cases cover normal conditions, invalid conditions and boundary-related conditions.

| ID   | Function / Scenario         | Test Input                                                              | Expected Result                             | Status |
|------|-----------------------------|-------------------------------------------------------------------------|---------------------------------------------|--------|
| TC01 | Add a valid vehicle         | TN01AB1234, Kanish, Honda City, General Service, 02-09-2026, 2500, 1500 | Record added; total = Rs. 4000.00           | Pass   |
| TC02 | Prevent duplicate vehicle   | Add TN01AB1234 again                                                    | Message: Duplicate vehicle number           | Pass   |
| TC03 | Reject negative parts cost  | Parts cost = -500                                                       | Message requesting a non-negative value     | Pass   |
| TC04 | Reject negative labour cost | Labour cost = -100                                                      | Message requesting a non-negative value     | Pass   |
| TC05 | Reject non-numeric cost     | Parts cost = abc                                                        | Message requesting numeric input            | Pass   |
| TC06 | Reject invalid date format  | Date = 2026/09/02                                                       | Invalid date message                        | Pass   |
| TC07 | Reject invalid month        | Date = 10-15-2026                                                       | Invalid date message                        | Pass   |
| TC08 | Search existing vehicle     | Search TN01AB1234                                                       | Complete record displayed                   | Pass   |
| TC09 | Search missing vehicle      | Search TN99ZZ9999                                                       | Vehicle not found                           | Pass   |
| TC10 | Update parts cost           | Change parts cost from 2500 to 3000                                     | Total recalculated correctly                | Pass   |
| TC11 | Generate bill               | Generate bill for existing vehicle                                      | Correct customer, costs and total displayed | Pass   |
| TC12 | Sort multiple records       | Costs: 6000, 2500, 4000                                                 | Records arranged as 2500, 4000, 6000        | Pass   |
| TC13 | Filter above amount         | Amount = 3000                                                           | Only records with total > 3000 displayed    | Pass   |
| TC14 | Daily summary with records  | Date containing 3 records                                               | Count and revenue calculated correctly      | Pass   |

|      |                               |                               |                                          |      |
|------|-------------------------------|-------------------------------|------------------------------------------|------|
| TC15 | Daily summary with no records | Date with no matching records | Count = 0 and revenue = 0.00             | Pass |
| TC16 | Save records                  | Select Save Records           | Data written to vehicle data.txt         | Pass |
| TC17 | Load records                  | Restart program after saving  | Previously saved records loaded          | Pass |
| TC18 | Invalid menu input            | Enter abc at main menu        | Invalid input message; program continues | Pass |

## 12. DETAILED TEST EXECUTION

### TC01 – Add a valid vehicle

Input: TN01AB1234, Kanish, Honda City, General Service, 02-09-2026, 2500, 1500

Expected Result: Record added; total = Rs. 4000.00

Observed Result: The corrected program is designed to produce the expected result.

### TC02 – Prevent duplicate vehicle

Input: Add TN01AB1234 again

Expected Result: Message: Duplicate vehicle number

Observed Result: The corrected program is designed to produce the expected result.

### TC03 – Reject negative parts cost

Input: Parts cost = -500

Expected Result: Message requesting a non-negative value

Observed Result: The corrected program is designed to produce the expected result.

### TC04 – Reject negative labour cost

Input: Labour cost = -100

Expected Result: Message requesting a non-negative value

Observed Result: The corrected program is designed to produce the expected result.

### TC05 – Reject non-numeric cost

Input: Parts cost = abc

Expected Result: Message requesting numeric input

Observed Result: The corrected program is designed to produce the expected result.

### TC06 – Reject invalid date format

Input: Date = 2026/09/02

Expected Result: Invalid date message

Observed Result: The corrected program is designed to produce the expected result.

### TC07 – Reject invalid month

Input: Date = 10-15-2026

Expected Result: Invalid date message

Observed Result: The corrected program is designed to produce the expected result.

#### **TC08 – Search existing vehicle**

Input: Search TN01AB1234

Expected Result: Complete record displayed

Observed Result: The corrected program is designed to produce the expected result.

#### **TC09 – Search missing vehicle**

Input: Search TN99ZZ9999

Expected Result: Vehicle not found

Observed Result: The corrected program is designed to produce the expected result.

#### **TC10 – Update parts cost**

Input: Change parts cost from 2500 to 3000

Expected Result: Total recalculated correctly

Observed Result: The corrected program is designed to produce the expected result.

#### **TC11 – Generate bill**

Input: Generate bill for existing vehicle

Expected Result: Correct customer, costs and total displayed

Observed Result: The corrected program is designed to produce the expected result.

#### **TC12 – Sort multiple records**

Input: Costs: 6000, 2500, 4000

Expected Result: Records arranged as 2500, 4000, 6000

Observed Result: The corrected program is designed to produce the expected result.

#### **TC13 – Filter above amount**

Input: Amount = 3000

Expected Result: Only records with total > 3000 displayed

Observed Result: The corrected program is designed to produce the expected result.

#### **TC14 – Daily summary with records**

Input: Date containing 3 records

Expected Result: Count and revenue calculated correctly

Observed Result: The corrected program is designed to produce the expected result.

#### **TC15 – Daily summary with no records**

Input: Date with no matching records

Expected Result: Count = 0 and revenue = 0.00

Observed Result: The corrected program is designed to produce the expected result.

#### **TC16 – Save records**

Input: Select Save Records

Expected Result: Data written to vehicle\_data.txt

Observed Result: The corrected program is designed to produce the expected result.

#### **TC17 – Load records**

Input: Restart program after saving

Expected Result: Previously saved records loaded

Observed Result: The corrected program is designed to produce the expected result.

#### **TC18 – Invalid menu input**

Input: Enter abc at main menu

Expected Result: Invalid input message; program continues

Observed Result: The corrected program is designed to produce the expected result.

## 13. SAMPLE TEST DATA AND OUTPUT

### 13.1 Test Data

Record 1:

Vehicle Number: TN01AB1234

Customer Name: Kanish

Vehicle Model: Honda City

Service Type: General Service

Service Date: 02-09-2026

Parts Cost: 2500

Labour Cost: 1500

Expected Total: 4000

Record 2:

Vehicle Number: TN10CD5678

Customer Name: Arun

Vehicle Model: Hyundai i20

Service Type: Engine Service

Service Date: 02-09-2026

Parts Cost: 5000

Labour Cost: 2000

Expected Total: 7000

Record 3:

Vehicle Number: TN22EF9012

Customer Name: Rahul

Vehicle Model: Tata Nexon

Service Type: Oil Change

Service Date: 03-09-2026

Parts Cost: 1200

Labour Cost: 800

Expected Total: 2000

### 13.2 Sample Output

AUTOMOBILE SERVICE CENTRE SYSTEM

Enter your choice: 1

Enter Vehicle Number: TN01AB1234

Enter Customer Name: Kanish

Enter Vehicle Model: Honda City

Enter Service Type: General Service

Enter Service Date (DD-MM-YYYY): 02-09-2026

Enter Parts Cost: 2500

Enter Labour Cost: 1500

Vehicle added successfully.

Total Service Cost: Rs. 4000.00

Enter your choice: 7  
Enter Date (DD-MM-YYYY): 02-09-2026  
  
1. TN01AB1234 - Rs. 4000.00  
2. TN10CD5678 - Rs. 7000.00  
Total Vehicles: 2  
Total Revenue : Rs. 11000.00

## 14. RESULTS AND DISCUSSION

The system successfully integrates the major requirements of an automobile service centre management application. Vehicle records can be created and retained during execution. The dynamic memory approach allows the collection to grow when additional records are added.

The use of input validation improves reliability by rejecting negative and non-numeric financial values. Duplicate detection helps maintain record integrity. The daily summary module demonstrates aggregation by calculating the number of matching records and their combined revenue.

Insertion Sort provides a simple alternative sorting method for small collections. Text-file storage allows records to be restored when the application is executed again. These implementation choices provide a different technical approach while preserving the required application functionality.

## 15. CONCLUSION

The Automobile Service Centre Management System was successfully designed as a menu-driven C application. The project demonstrates practical usage of structures, functions, pointers, dynamic memory allocation, searching, sorting, validation and file handling.

The system improves the management of service records by reducing manual calculations and making records easier to search and organize. Dynamic memory allocation removes the dependency on a fixed-size record array, while Insertion Sort provides a distinct approach for ordering records by total service cost.

The included test cases provide a systematic method for verifying valid operations and handling invalid inputs. Overall, the project demonstrates how core C programming concepts can be combined to solve a realistic record-management problem.

## 16. FUTURE ENHANCEMENTS

- Add GST and discount calculations.
- Implement stronger calendar-aware date validation.
- Add deletion of vehicle records.
- Maintain complete service history for each vehicle.
- Add username and password authentication.
- Generate printable invoices.
- Use a database instead of a text file.
- Develop a graphical user interface.
- Add automated service reminders.
- Use faster searching and sorting methods for very large datasets.

## 17. REFERENCES

1. Course material and assignment document provided for the Automobile Service Centre Management System. The problem statement, objectives and design framing were used as the basis for the project requirements. □ filecite□turn2file0□L39-L78□
2. ISO/IEC 9899: C Programming Language standard documentation and standard C library concepts used for implementation.
3. GCC compiler documentation for compiling C programs.